

Laboratorio 2

Este es el laboratorio práctico para la comprensión de las instrucciones SIMD en el rendimiento de un programa.

Para este laboratorio, necesitará:

- Una computadora con procesador x86-64.
- Sistema Operativo Linux con su entorno de desarrollo.

Este laboratorio está diseñado para ejecutarse de forma individual.

Objetivo

Este laboratorio tiene como objetivo comprender, mediante un ejercicio práctico, los efectos de utilizar instrucciones vectoriales.

Preparación del entorno

En el sistema de desarrollo (computadora personal o sistema empujado), verifique que tiene instalado el compilador de C:

```
gcc --version
```

Si el comando no existe:

```
sudo apt update  
sudo apt install build-essential
```

Para todos los ejercicios, debe usar *intrinsics* de AVX2. Puede utilizar la plantilla proporcionada en el TecDigital como punto de partida.

Ejercicio A

Realice una multiplicación de dos vectores de punto flotante de tamaño 1024, tal que se cumpla:

$$\mathbf{c} = \mathbf{a}\mathbf{b}$$

donde

$$c_i = a_i b_i, \forall i \in \{0, \dots, 1023\}$$

Ejercicio B

Realice una reducción de un vector de punto flotante de tamaño 1024, tal que se cumpla:

$$c = \sum_{i=0}^{1023} a_i$$

Ejercicio C

Utilizando las instrucciones de los ejercicios A y B, complete el algoritmo de multiplicación de matrices.

Ejercicio D

Contraste los tiempos de ejecución de la versión no vectorizada, con la versión vectorizada.

Entregable

Es un ejercicio para realizar en clase. Debe incluirse dentro de un repositorio de Git donde se manejen los laboratorios de clase. Se debe subir el enlace del repositorio a través del TecDigital.

Laboratorio 3

Este es el laboratorio práctico para la comprensión de cómo afecta la organización de un computador en la forma de ser programado de forma óptima.

Para este laboratorio, necesitará:

- Una computadora con procesador x86-64.
- Sistema Operativo Linux con su entorno de desarrollo.
- Los códigos del TecDigital

Este laboratorio está diseñado para ejecutarse de forma individual.

Objetivo

Este laboratorio tiene como objetivo comprender, mediante un ejercicio práctico, los efectos de utilizar alineamiento, multihilos y afinidad.

Preparación del entorno

En el sistema de desarrollo (computadora personal o sistema empotrado), verifique que tiene instalado el compilador de C:

```
gcc --version
```

Si el comando no existe:

```
sudo apt update  
sudo apt install build-essential
```

Para todos los ejercicios, debe usar *intrinsics* de AVX2. Puede utilizar la plantilla proporcionada en el TecDigital como punto de partida.

Ejercicio A

Compile y ejecute el código de `cpu-affinity` y `cpu-naive`. Por ejemplo:

```
time ./cpu-affinity
```

El resultado del comando son distintos tiempos. Tome nota del tiempo *real*.

Luego modifique el número de hilos desde 1 hasta el máximo disponible en su computadora. Luego realice un gráfico con los tiempos *real*. Luego, explique con lo visto en clase y sobre evaluación de rendimiento cómo es la escalabilidad, de cuánto es la proporción de código paralelo con respecto al total y la eficiencia.

Ejercicio B

Compile y ejecute el código de `matmul_tiled.openmp` y `softmax.openmp`.

Luego modifique el número de hilos desde 1 hasta el máximo disponible en su computadora, a través del argumento de la aplicación:

```
./softmax.openmp 2
```

donde 2 es el número de hilos.

Luego realice un gráfico con los tiempos reportados por la aplicación. Luego, explique con lo visto en clase y sobre evaluación de rendimiento cómo es la escalabilidad, de cuánto es la proporción de código paralelo con respecto al total y la eficiencia.

Ejercicio C

Siguiendo el código realizado en el Laboratorio 2 del Capítulo 1, vectorice el código de ambos programas y repita las mediciones, como hizo en los ejercicios A y B.

Entregable

Es un ejercicio para realizar en clase. Debe incluirse dentro de un repositorio de Git donde se manejen los laboratorios de clase. Se debe subir el enlace del repositorio a través del TecDigital.